

Lab-2-Report

Note

实验代码见本[仓库](#)。

实验介绍

实验目的

本实验围绕 MIPS 五段流水线的执行过程进行设计与验证，目标是通过自行实现流水线模拟器，加深对流水线基本概念、各流水段功能、数据冲突、控制冲突以及定向路径作用的理解。

具体目标如下：

1. 理解 MIPS 五段流水线中 `IF`、`ID`、`EX`、`MEM`、`WB` 各阶段的职责和状态传递方式。
2. 掌握 RAW 数据相关、分支控制相关等冲突对流水线性能的影响。
3. 通过开关定向路径，对比有无转发时流水线停顿数量和 CPI 的变化。
4. 设计至少三类测试程序，分别展示无冲突、RAW 冲突和分支跳转场景。
5. 使用图形界面直观展示程序执行过程，包括流水段状态、寄存器状态、内存状态、冲突事件和性能统计。

实验内容与要求

根据实验文档，本实验中自行设计的模拟器需要满足以下功能要求：

1. 能够模拟 MIPS 的五段流水线执行过程。
2. 支持图形交互或命令交互。本实验选择图形交互方式实现。
3. 支持单步执行、执行到断点和执行到程序结束。
4. 支持查看流水线各阶段状态和寄存器状态。
5. 提供是否使用定向路径的功能选项。
6. 提供程序执行后的性能统计分析。
7. 按照 MIPS 语法，至少支持 `load`、`store`、`add`、`beqz` 操作。
8. 支持直接输入汇编程序或通过文件载入程序。
9. 至少设计三类代码组合，用于展示无冲突、RAW 冲突和分支跳转现象。

实验环境

- 操作系统：Linux
- Python：3.10 及以上
- 包管理工具：`uv`
- 前端运行环境：Node.js 18 及以上、`npm`
- 后端框架：FastAPI
- 前端框架：React + TypeScript + Vite
- 测试工具：pytest、Vitest、Playwright

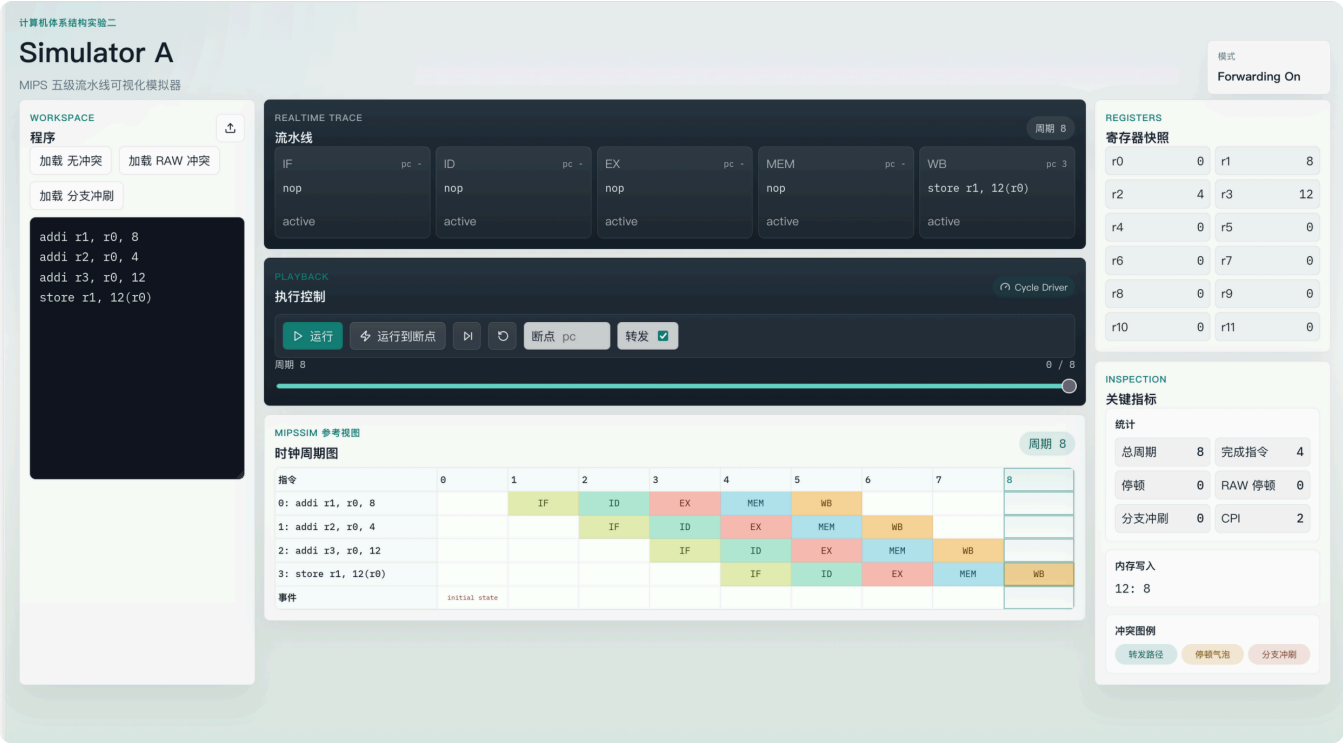
总体设计

本实验采用前后端分离架构。后端负责流水线模拟的核心逻辑，前端负责程序输入、运行控制和可视化展示。

后端由 `sim_a/` 和 `server/` 两部分组成。`sim_a/` 实现指令解析、流水线状态推进、冒险检测、寄存器和内存更新、统计信息生成；`server/` 使用 FastAPI 提供 HTTP 接口，前端提交汇编程序后，后端返回完整的逐周期执行 trace。

前端由 `frontend/` 实现，采用 React 组件化组织界面。用户可以在页面中选择示例程序、上传 `.asm` 文件、编辑汇编源码，并通过“运行”“运行到断点”“单步”“复位”等按钮观察程序执行过程。

与单纯命令行输出相比，本实验的设计重点是把“每个周期发生了什么”以图形方式呈现出来。因此，页面中除五级流水线状态面板外，还增加了时钟周期图。时钟周期图以“指令 x 周期”的形式展示每条指令在各周期所处的流水段，是分析冲突和停顿现象的主要依据。



ISA 设计与扩展

实验文档要求模拟器至少支持 `load`、`store`、`add`、`beqz` 四类操作。当前实现支持的指令如下：

指令	格式	功能
<code>load</code>	<code>load rd, offset(rs)</code>	从内存地址 $R[rs] + offset$ 读取数据到 <code>rd</code>
<code>store</code>	<code>store rs, offset(rd)</code>	将 <code>rs</code> 中的数据写入内存地址 $R[rd] + offset$
<code>add</code>	<code>add rd, rs, rt</code>	将 <code>rs</code> 与 <code>rt</code> 相加，结果写入 <code>rd</code>
<code>addi</code>	<code>addi rd, rs, imm</code>	将 <code>rs</code> 与立即数相加，结果写入 <code>rd</code>
<code>beqz</code>	<code>beqz rs, label</code>	当 <code>rs</code> 的值为 0 时跳转到标签 <code>label</code>

其中 `addi` 是在基础要求之外增加的指令。增加该指令的原因是：如果只有 `load`、`store`、`add`、`beqz`，示例程序很难在完全由汇编代码驱动的前提下生成非零初始值，容易依赖后端预置寄存器或内存。扩展 `addi` 后，程序可以自行完成寄存器赋值，使实验现象完全由指令序列产生，更便于在报告中说明程序行为和状态变化之间的关系。

后端设计与实现

指令解析

指令解析模块将用户输入的汇编文本转换为内部 `Instruction` 对象。解析过程会忽略空行，并支持标签定义。对于分支指令，标签会被解析为目标指令位置，供流水线执行时修改 `PC`。

解析器还会记录每条指令的源寄存器和目的寄存器。该信息用于后续 RAW 冲突检测。例如，`load r2, 0(r0)` 的目的寄存器为 `r2`，紧随其后的 `add r3, r2, r1` 会读取 `r2`，因此二者之间存在 RAW 数据相关。

五段流水线推进

模拟器显式维护 `IF`、`ID`、`EX`、`MEM`、`WB` 五个流水段寄存器。每推进一个周期，模拟器按流水线时序完成写回、访存、执行、译码和取指相关操作，并生成一个 trace frame。

每个 trace frame 包含以下信息：

- 当前周期号；

- 当前 `PC`；
- 五个流水段中的指令；
- 通用寄存器状态；
- 内存状态；
- 本周期发生的事件；
- 当前累计统计信息。

前端的单步执行并不是重新运行一条指令，而是在后端生成的 trace 序列中移动当前周期下标。因此，用户可以稳定地回看每一个周期的流水线状态。

RAW 冲突与定向路径

模拟器支持对 RAW 数据相关进行检测。当一条指令需要读取前序指令尚未写回的目的寄存器时，会根据定向路径开关决定处理方式：

1. 关闭定向路径时，模拟器通过插入停顿等待数据写回。
2. 开启定向路径时，模拟器尽量从后续流水段转发数据，从而减少部分停顿。

这种设计可以直接观察定向路径对流水线性能的影响。对于 RAW 冲突示例，关闭转发时停顿次数会增加；开启转发后，部分相关可以通过旁路解决，CPI 相应降低。

分支冲刷

`beqz` 指令在条件成立时会修改 `PC`，并清空错误路径上已经进入流水线的指令。前端在时钟周期图和事件列表中展示分支冲刷现象，便于分析控制相关对流水线执行的影响。

性能统计

后端在执行过程中统计以下指标：

- 总周期数；
- 退役指令（Instruction Retired）数；
- 停顿次数；
- RAW 停顿次数；
- 分支冲刷次数；
- CPI。

CPI 的计算公式为：

$$\text{CPI} = \text{总周期数} / \text{退役指令数}$$

Unkown

当前前端显示 CPI 时，如果结果不是整数，会保留三位小数，便于比较不同设置下的性能差异。

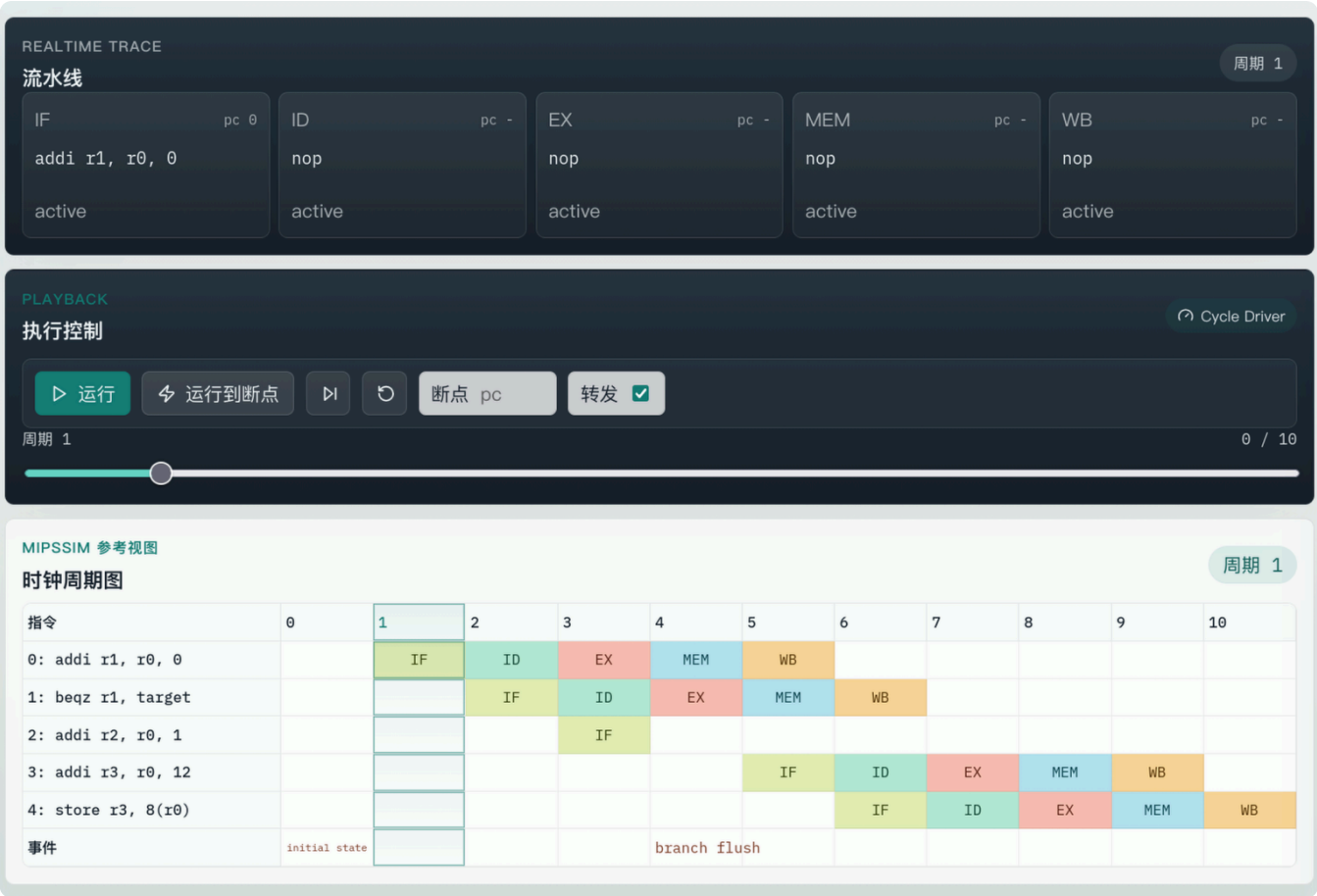
前端设计与实现

页面布局

前端页面采用三栏工作台布局：

1. 左侧为程序区，提供示例程序选择、文件上传和汇编代码编辑。
2. 中间为核心执行区，展示五级流水线状态、运行控制条和时钟周期图。
3. 右侧为系统状态区，集中展示统计信息、内存写入和冲突图例。

这种布局参考了传统 MIPS 流水线模拟器的窗口组织方式，但将多个信息窗口整合到一个页面中，减少滚动和切换成本。



交互功能

前端支持以下操作：

- 使用内置示例程序；
- 上传本地 `.asm` 或 `.txt` 文件；
- 直接编辑汇编源码；
- 运行到程序结束；
- 运行到指定断点；
- 单周期执行；
- 复位到第 0 个周期；
- 开启或关闭定向路径。

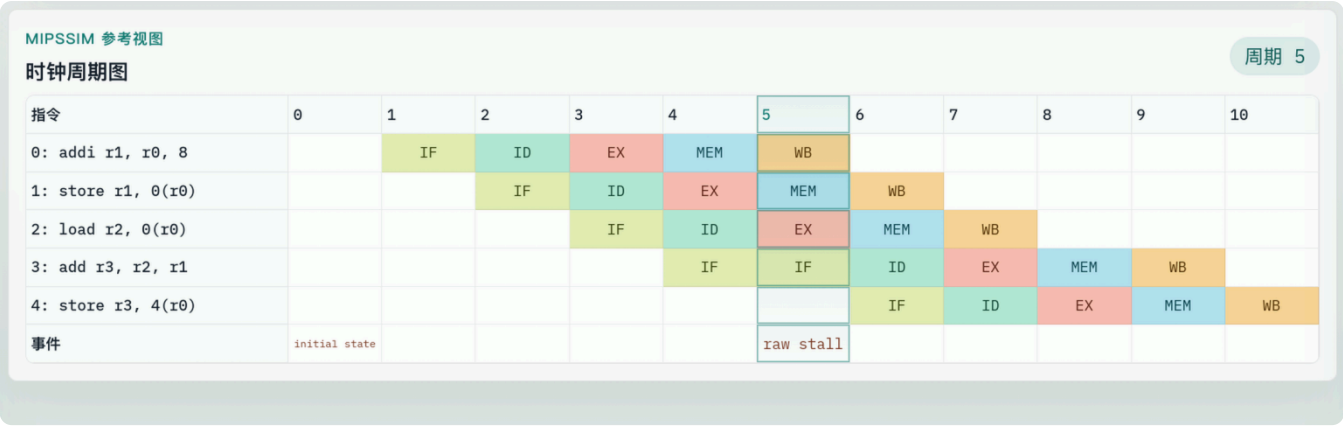
按钮和标签已尽量采用中文，以符合课程实验报告和演示习惯；`IF`、`ID`、`EX`、`MEM`、`WB` 等流水线阶段名称保留英文缩写，因为它们是计算机体系结构课程中的常用记号。

时钟周期图

时钟周期图是本实验前端的核心可视化模块。该图每一行对应一条汇编指令，每一列对应一个时钟周期，单元格中显示该指令在对应周期所处的流水段。

图中使用不同颜色区分 `IF`、`ID`、`EX`、`MEM`、`WB`，并高亮当前周期。通过该图可以直观看到：

- 指令如何逐周期进入流水线；
- 停顿发生在哪个周期；
- 转发开启后停顿是否减少；
- 分支跳转后错误路径指令如何被冲刷；
- 程序整体执行到完成所需的周期数。



实验结果

环境配置

首先在项目根目录使用 `uv` 创建 Python 虚拟环境并安装后端依赖：

```
uv sync --group dev
```

bash

然后进入前端目录安装 Node.js 依赖：

```
cd frontend
npm install
```

bash

若需要运行端到端测试，还需要安装 Playwright 使用的 Chromium：

```
npx playwright install chromium
```

bash

启动系统

后端启动命令如下：

```
.venv/bin/uvicorn server.app:app --host 127.0.0.1 --port 8765
```

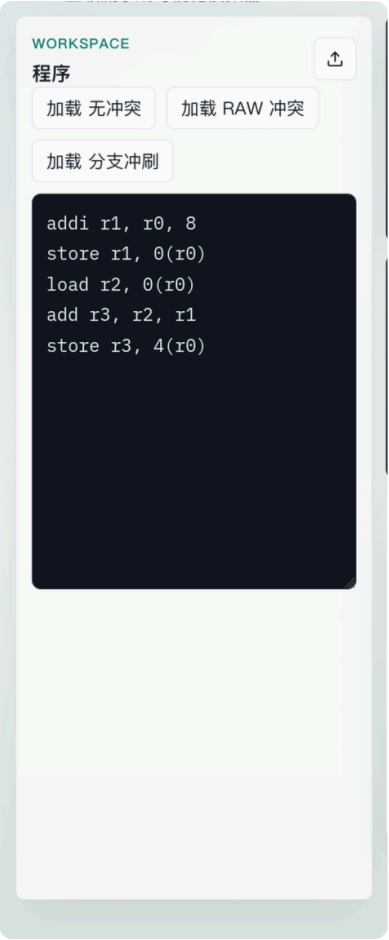
bash

前端启动命令如下：

```
cd frontend
npm run dev -- --host 127.0.0.1
```

bash

启动后，在浏览器访问前端页面。用户可以直接选择内置示例程序，也可以上传自己的汇编程序进行模拟。



加载并运行示例程序

实验中分别加载无冲突、RAW 冲突和分支跳转三类程序。每类程序先运行到结束，记录总体统计信息；然后使用单步按钮逐周期观察流水线状态；最后在需要时切换定向路径开关，比较停顿次数和 CPI 的变化。

在观察过程中，重点记录以下内容：

- 每条指令进入 **IF** 的周期；
- RAW 冲突出现的位置；
- 停顿周期是否插入；
- 分支成立时是否发生冲刷；
- 寄存器和内存状态是否符合程序语义；
- CPI 是否随停顿数量变化。

测试代码组合与现象分析

无冲突场景

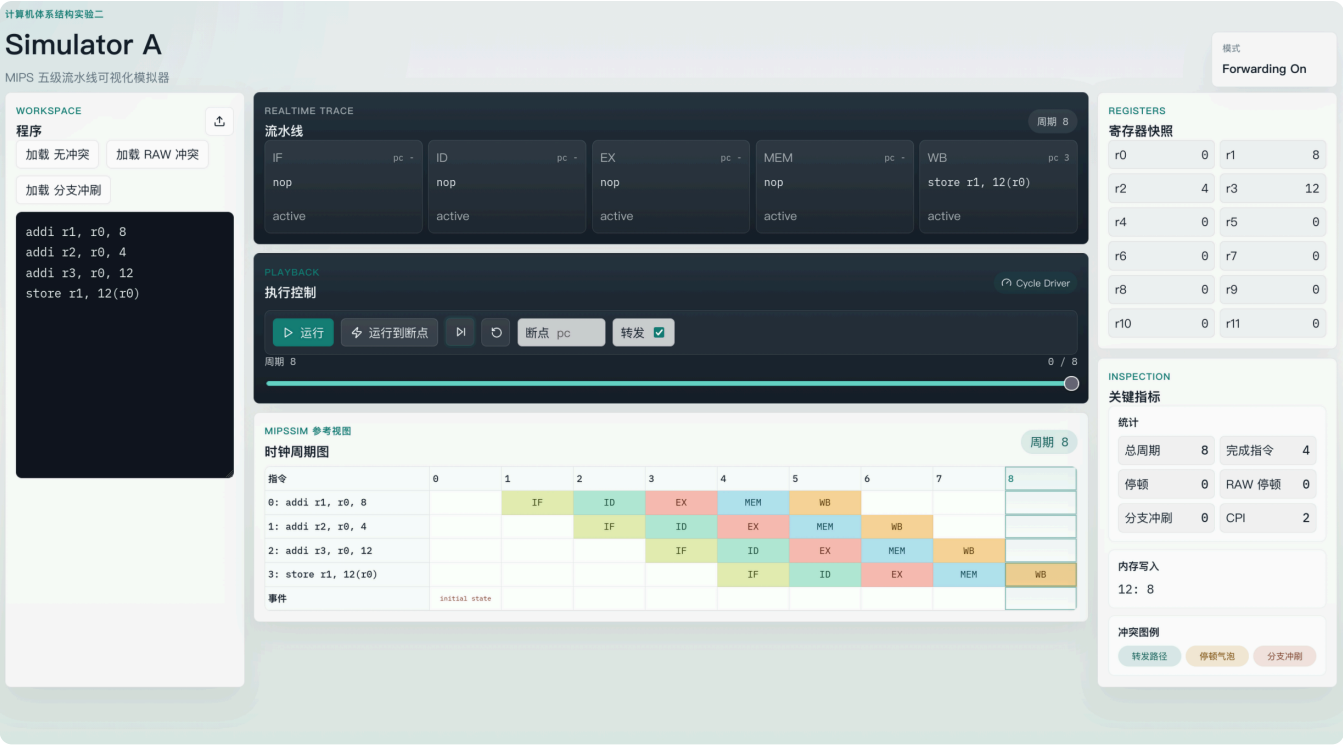
测试程序如下：

```
addi r1, r0, 8
addi r2, r0, 4
addi r3, r0, 12
store r1, 12(r0)
```

Unknown

该程序先通过三条 **addi** 指令给寄存器赋值，再执行一次内存写入。该示例主要用于验证基础流水线推进、寄存器写回和内存写入功能。由于程序中没有紧邻的 load-use 相关，也没有分支跳转，因此该程序的执行过程相对平稳，适合作为模拟器基本正确性的验证用例。

由下图可以看到，一切工作正常。



RAW 冲突场景

测试程序如下：

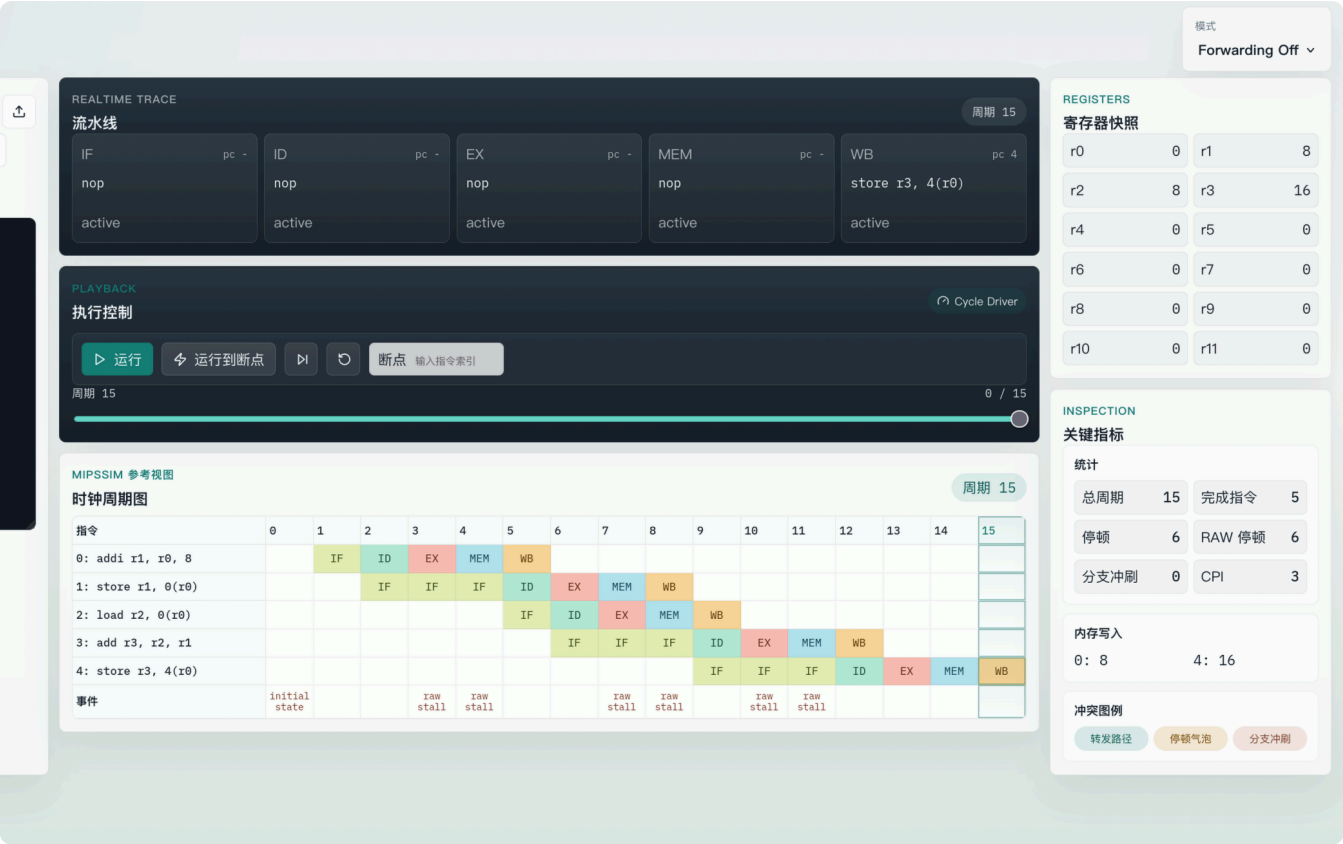
```
addi r1, r0, 8
store r1, 0(r0)
load r2, 0(r0)
add r3, r2, r1
store r3, 4(r0)
```

Unknown

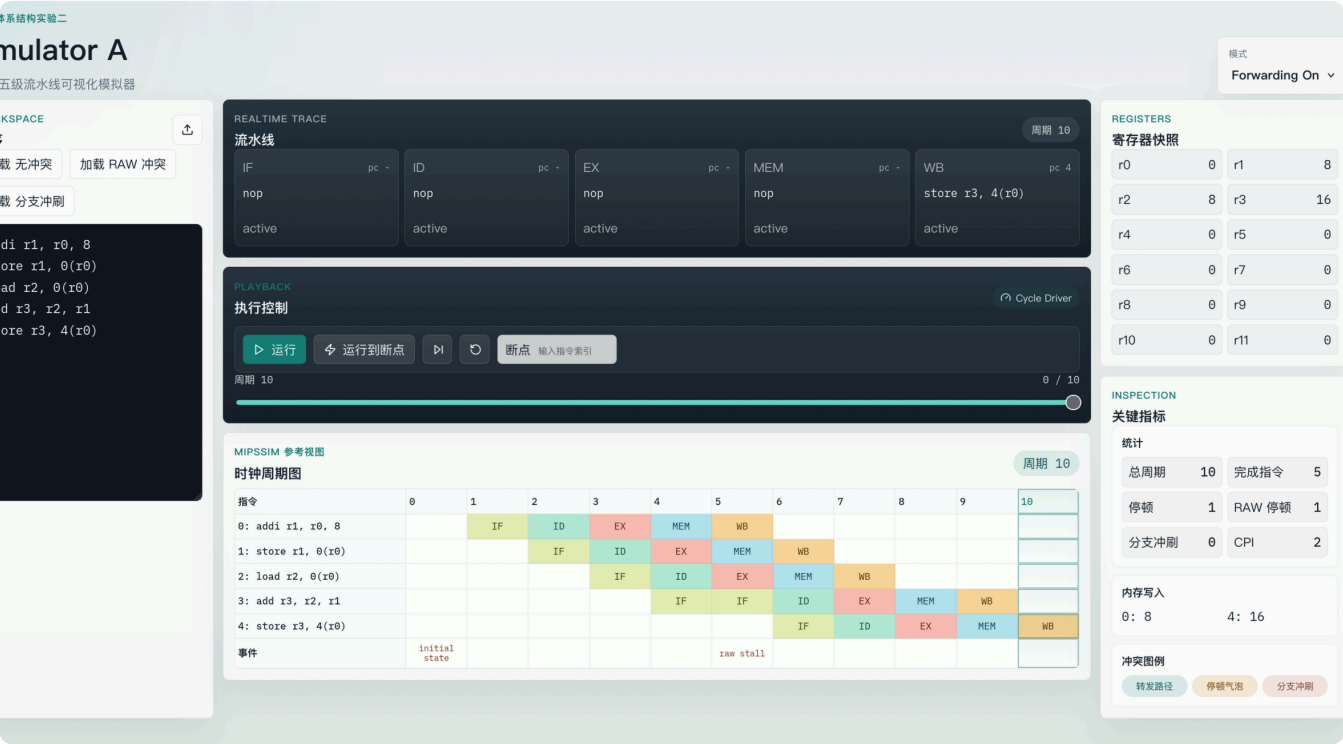
该程序先将 `r1` 初始化为 8，并写入内存地址 0；随后 `load r2, 0(r0)` 从内存读取该值，紧接着 `add r3, r2, r1` 需要使用 `r2`。由于 `r2` 是前一条 `load` 指令的结果，后续 `add` 指令在数据可用之前不能直接完成执行，因此形成 RAW 数据相关。

当关闭定向路径时，模拟器需要插入更多停顿等待数据写回；当开启定向路径时，可以通过转发减少部分等待周期。该示例用于观察定向路径对流水线性能改善。

关闭定向路径：



开启定向：



可以看到，本例中，`load` 与紧随其后的 `add` 形成 RAW 冲突；关闭定向路径时出现了更多停顿（6次），而开启定向路径后 RAW 停顿次数减少到1次；开启定向后，CPI 随着停顿减少从3下降到了2。但是无论是否开启定向，`r3` 的最终值均等于 `r2 + r1`。说明这一场景下，模拟器工作正常~

分支跳转场景

测试程序如下：

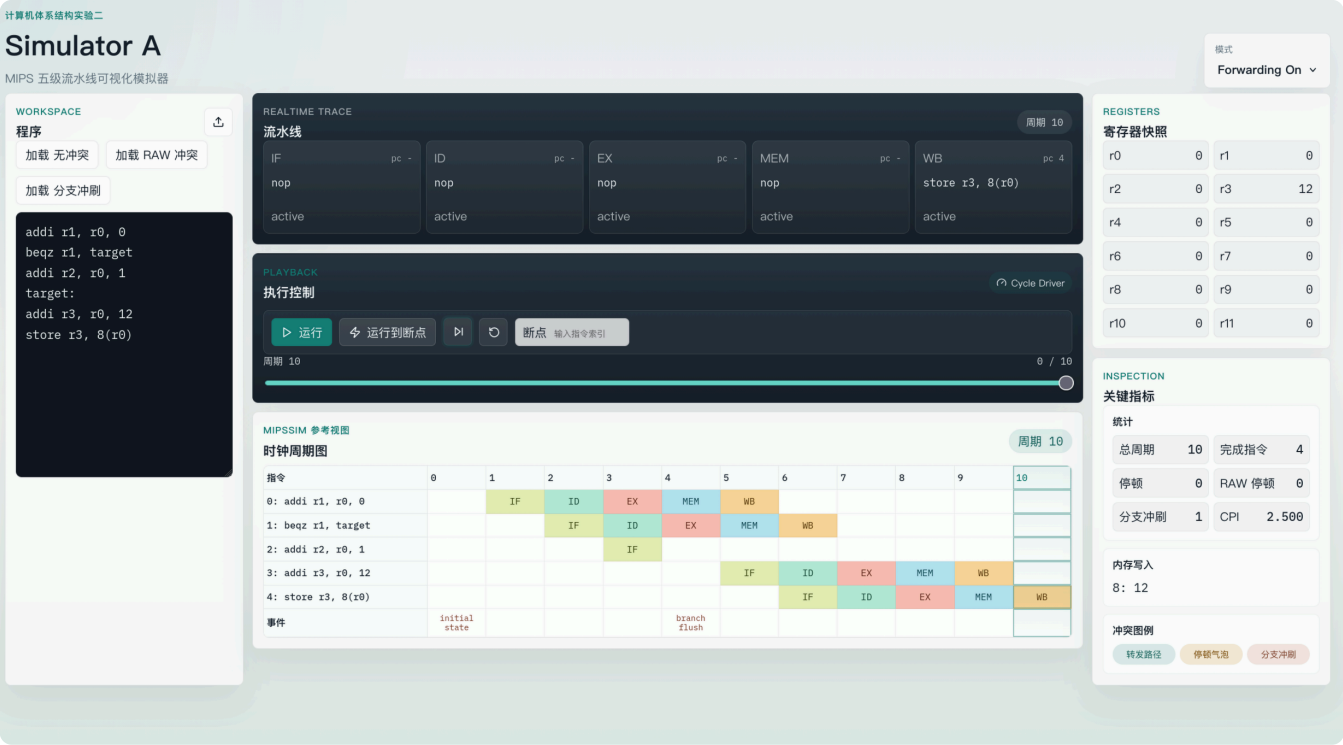
```
addi r1, r0, 0
beqz r1, target
addi r2, r0, 1
```

Unknown


```
target:
addi r3, r0, 12
store r3, 8(r0)
```

该程序首先将 `r1` 置为 0，随后执行 `beqz r1, target`。由于 `r1` 的值为 0，分支条件成立，程序跳转到 `target` 标签处继续执行。因此，位于分支后的 `addi r2, r0, 1` 属于错误路径指令，应被冲刷或不产生最终影响。

该示例主要用于展示控制相关带来的流水线冲刷现象。



`beqz` 判断条件成立后，`PC` 能够正常跳转到 `target`；错误路径上的 `addi r2, r0, 1` 并没有影响最终状态；且时钟周期图中能观察到分支冲刷事件，统计信息中分支冲刷次数已增加。说明模拟器在这一场景工作正常。

问题与解决方法

示例程序无法产生明显的非零寄存器值

最初版本中，示例程序主要依赖内存读取产生寄存器变化。如果没有预置内存状态，很多寄存器值会一直保持为 0，导致前端展示效果不明显，也不利于说明程序执行结果。

解决方法是在 ISA 中扩展 `addi` 指令，并修改示例程序，使其通过汇编指令主动完成赋值。这样寄存器和内存变化都由程序本身产生，报告中的分析也更符合“程序决定机器状态”的原则。

页面信息较多，时钟周期图容易被挤到下方

初始界面中流水线面板和控制面板高度较大，右侧统计、内存和冲突图例也较分散，导致用户需要滚动页面才能看到时钟周期图。

解决方法是压缩中间区域高度，将统计、内存和冲突图例整合到同一系统状态面板中，并让时钟周期图与中间主工作区宽度对齐。这样在常见屏幕尺寸下可以更容易同时观察流水线状态、控制按钮和周期图。

时钟周期图中的事件文字可能超出单元格

时钟周期图中 `initial state` 等事件文字较长，在表格单元格较窄时可能出现文字溢出，影响观感。

解决方法是对该类事件单元格单独降低字号，并减小行高，使表格在保持信息完整的同时避免文字撑开布局。

测试与验证

本项目进行了后端、前端和端到端三个层面的验证。

后端测试命令如下：

```
uv run python -m pytest tests/test_instruction.py tests/test_pipeline.py tests/test_hazards.py
tests/test_api.py -q
```

bash

前端单元测试命令如下：

```
cd frontend
npm run test
```

bash

前端构建检查命令如下：

```
cd frontend
npm run build
```

bash

端到端测试命令如下：

```
cd frontend
npm run test:e2e
```

bash

测试覆盖内容包括：

- `addi`、`load`、`store`、`add`、`beqz` 的解析；
- 五段流水线的逐周期推进；
- RAW 冲突检测和停顿插入；
- 定向路径开启/关闭后的行为差异；
- FastAPI 接口返回 trace 的正确性；
- 前端时钟周期图、系统状态面板和主交互流程。

分析与总结

实验分析

从三类示例程序的执行结果可以看出，模拟器能够正确展示 MIPS 五段流水线的基本行为。

无冲突示例中，指令按顺序流过五个流水段，寄存器和内存更新结果符合汇编程序语义。该示例验证了模拟器的基础执行路径。

RAW 冲突示例中，`load` 后紧接使用目标寄存器的 `add` 指令会触发数据相关。关闭定向路径时，流水线需要插入停顿；开启定向路径后，部分数据可通过转发获得，停顿次数减少，CPI 得到改善。该现象说明定向路径能够降低数据相关对流水线性能的影响。

分支跳转示例中，`beqz` 条件成立后，程序跳转到目标标签，错误路径指令不会影响最终状态。时钟周期图中可以观察到分支冲刷事件，说明控制相关会使流水线丢弃已经进入错误路径的指令，从而带来额外性能开销。

总体而言，时钟周期图比单独展示当前流水段更适合分析实验现象。它不仅能显示某一周期的流水线状态，还能展示整段程序的时间展开过程，因此适合作为实验报告中说明 RAW 冲突、停顿和分支冲刷的主要截图材料。

实验总结

本实验完成了 Simulator A 的设计与实现。模拟器支持 MIPS 五段流水线的基本执行过程，能够通过 Web 页面展示流水线状态、寄存器状态、内存状态、冲突信息和性能统计，并支持单步执行、运行到断点、运行到结束和定向路径开关。

在实现过程中，最重要的改进是将示例程序的初始赋值逻辑从后端预置状态转移到汇编程序本身。通过扩展 `addi` 指令，测试代码能够自主生成非零寄存器值，使程序语义更加完整，也让实验现象更容易解释。

通过对无冲突、RAW 冲突和分支跳转三类程序的测试，可以看到流水线技术虽然能够提高指令吞吐率，但数据相关和控制相关仍会造成停顿或冲刷。定向路径可以缓解部分数据冲突，但不能完全消除所有流水线开销。该实验帮助我从实现角度理解了课堂中关于流水线性能分析的内容，也为后续使用或对比开源模拟器 B 奠定了基础。